Methoden

Einführung in die Programmierung 1
TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- Eingabe von der Konsole
- Scanner-Klasse
- Anwendungen der Scanner-Klasse

Was wird in diesem Foliensatz besprochen?

- Idee von Unterprogrammen
- Methoden in Java
- Parameter
- Rückgabe
- Überladen

Was lernen Sie kennen?

- (Statische) Methoden in Java
- Parameterübergabe in Java
- Rückgabe von Werten in Java
- Überladen von Methoden in Java

Motivation

Bisher gesamter Programmcode in der main-Methode

- Bei größeren Programmen wird das unübersichtlich und umständlich
- Einige Teile werden sehr ähnlich sein und immer wieder im Programm vorkommen

Programm sollte aufgeteilt werden

- Übersichtlicher
- Verständlicher

Unterprogramm

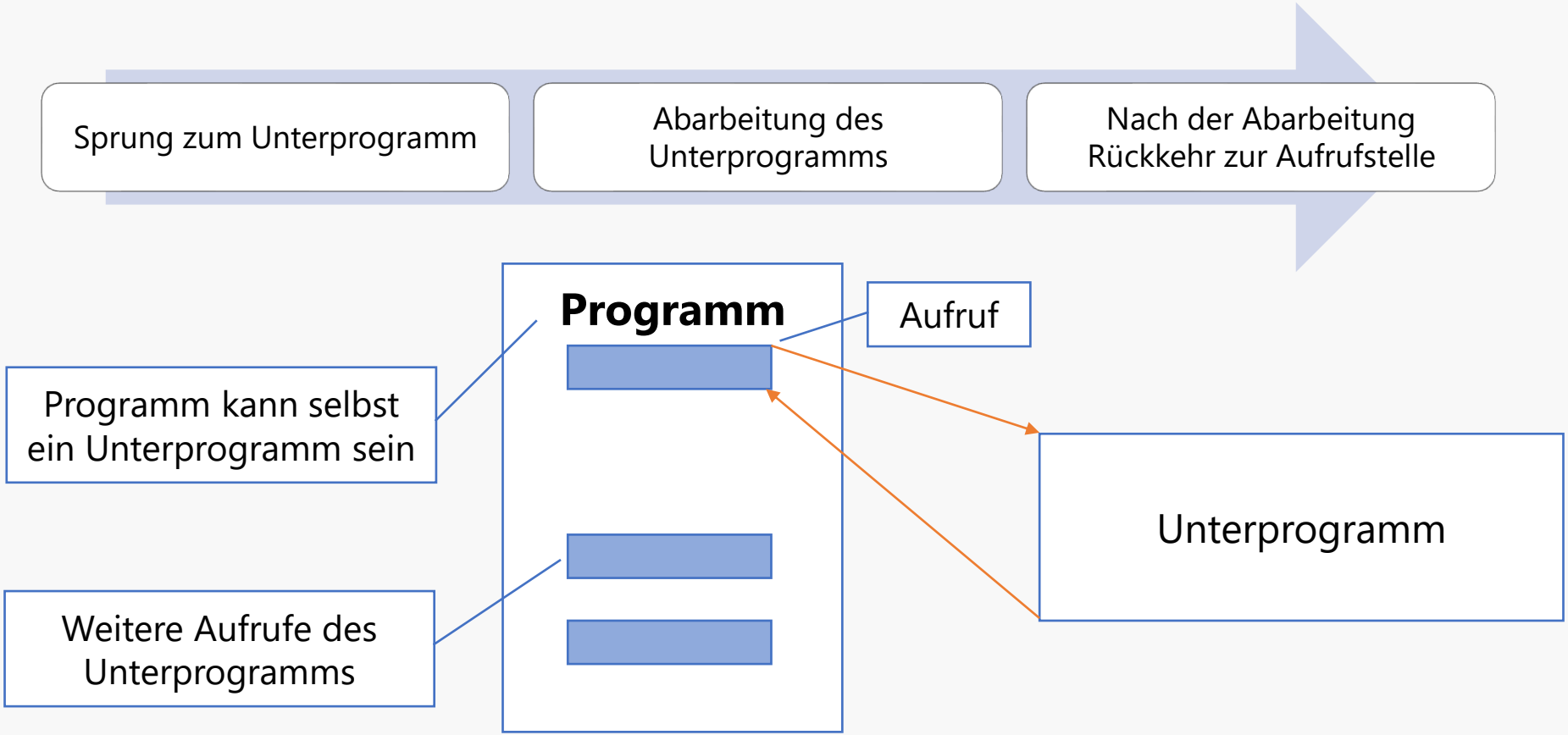
Teil eines Programms

Ist eine **isolierbare**, oft an **mehreren Stellen** im Programm **identisch** auszuführende Tätigkeit

Wird durch einen Bezeichner identifiziert

Weiterer Mechanismus zum Steuern des Programmablaufs (neben Verzweigungen und Schleifen)

Aufruf eines Unterprogramms



Vorteile von Unterprogrammen

Wiederverwendung

- Einmal Code schreiben
- Mehrfach an vielen Stellen mit unterschiedlichen Inputs (Argumenten) verwenden

Abstraktion

- Ablauf muss nicht genau bekannt sein
- Für den Aufruf nur wichtig
  - Welcher Input (Argumente) kann übergeben werden?
  - Welche Auswirkung hat der Aufruf (Output, Rückgabewert)?

Unterprogramme in Java

Werden als Methoden bezeichnet

Werden in einer Klasse implementiert

- Sind kein eigenständiger Teil (wie z. B. in C)

Methoden in Java

Statische Methoden	• Gehören zur Klasse • Werden in diesem Foliensatz besprochen
Objektbezogene Methoden	• Gehören zu einem Objekt der Klasse • Werden bei der Objektorientierung (EP2) besprochen

Aufbau von Methoden

Methodenkopf	• Modifikatoren, Rückgabetyp, Methodenname und eventuell Parameter
Methodenrumpf	• Enthält Variablendeklarationen und Anweisungen

Beispiel (einfache Methode)

```java
public class MethodTest1 {

    private static void hello() {
        System.out.println("In hello");
    }

    public static void main(String[] args) {
        System.out.println("In main");
        hello();
        System.out.println("In main");
        hello();
        System.out.println("In main");
    }

}
```

```
In main
In hello
In main
In hello
In main
```

Anatomie einer statischen Methode

Modifikator `static` = Methode gehört zur Klasse

Rückgabetyp `void` = keine Rückgabe

Name der Methode und Klammern ()

```
private static void hello() {
    System.out.println("In hello");
}
```

Modifikator `private` = Zugriff auf Methode hello nur in der Klasse erlaubt

Code, der beim Aufruf `hello();` ausgeführt wird

Beispiel (einfache Methode)

```java
public class MethodTest2 {

    private static void printLine() {
        for (int i = 0; i < 15; i++) {
            System.out.print("-");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        printLine();
        System.out.println("Hello World!");
        printLine();
        System.out.println("Hello EP1!");
        printLine();
    }

}
```

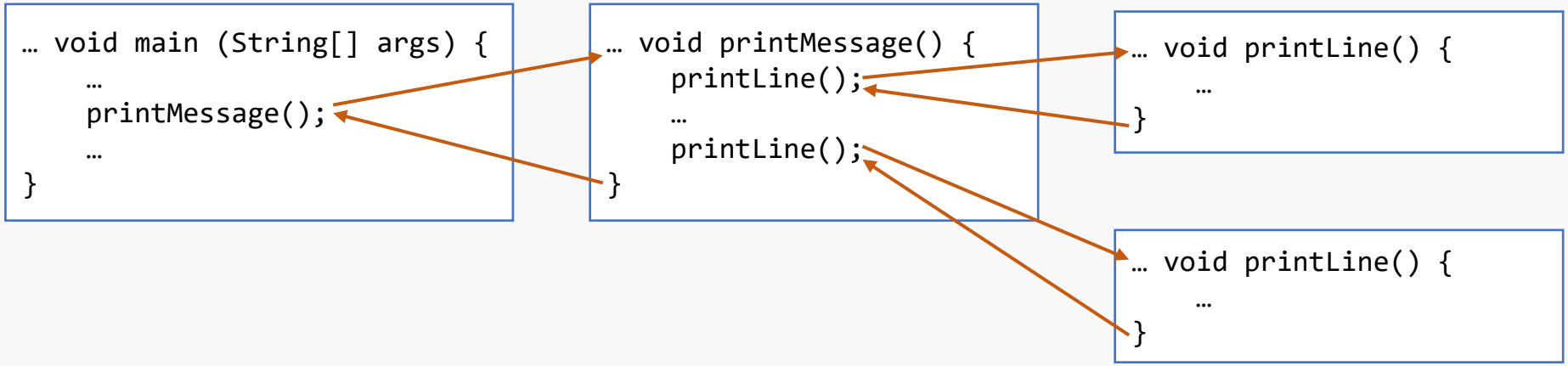
```
---------------
Hello World!
---------------
Hello EP1!
---------------
```

Beispiel (mehrere einfache Methoden)

```java
public class MethodTest3 {

    private static void printLine() {
        for (int i = 0; i < 15; i++) {
            System.out.print("-");
        }
        System.out.println();
    }

    private static void printMessage() {
        printLine();
        System.out.println("Hello World!");
        printLine();
    }

    public static void main(String[] args) {
        System.out.println("In main");
        printMessage();
        System.out.println("In main");
    }

}
```

```
In main
---------------
Hello World!
---------------
In main
```

Beispiel (Ablauf)



Vergleich (alles in main, mit Methoden)

```java
public class TestWithoutMethods {

    public static void main(String[] args) {
        System.out.println("In main");
        for (int i = 0; i < 15; i++) {
            System.out.print("-");
        }
        System.out.println();
        System.out.println("Hello World!");
        for (int i = 0; i < 15; i++) {
            System.out.print("-");
        }
        System.out.println();
        System.out.println("In main");
    }

}
```

Duplizierter Code

```java
public class MethodTest3 {

    private static void printLine() {
        for (int i = 0; i < 15; i++) {
            System.out.print("-");
        }
        System.out.println();
    }

    private static void printMessage() {
        printLine();
        System.out.println("Hello World!");
        printLine();
    }

    public static void main(String[] args) {
        System.out.println("In main");
        printMessage();
        System.out.println("In main");
    }

}
```

Methodenname

Name

- Sollte ausdrücken, was die Methode leistet
- Kann mit einem Verb beginnen
  - Abhängig von der Aufgabe
- Wird üblicherweise klein geschrieben
- Falls der Name aus mehreren Worten besteht, sollten die Folgeworte groß geschrieben werden (Konvention)

Beispiele

- **random, abs, max, min, sin, cos, sqrt …**
- **printHeader, createTable …**

Parameter

- Werte, die beim Aufruf an eine Methode übergeben werden
- Wir unterscheiden

Formale Parameter	• Werden im Methodenkopf angegeben • Werden innerhalb der Methode wie Variablen behandelt
Tatsächliche Parameter (Argumente)	• Beim Aufruf der Methode wird ein formaler Parameter durch je einen tatsächlichen Parameter initialisiert • Hinweis: In der deutschen Literatur werden diese Parameter auch sehr oft als aktuelle Parameter bezeichnet!

Beispiel (einfache Methode mit Parameter)

```java
public class MethodTest4 {

    private static void printLine(int length) {
        for (int i = 0; i < length; i++) {
            System.out.print("-");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        printLine(10);
        System.out.println("Hello World!");
        printLine(15);
        System.out.println("Hello EP1!");
        printLine(20);
    }

}
```

Formaler Parameter length

Tatsächlicher Parameter 10

```
----------
Hello World!
---------------
Hello EP1!
--------------------
```

Beispiel (Methode mit mehreren Parametern)

```
public class ParameterTest {

    private static void printMax(int a, int b) {
        if (a > b) {
            System.out.println(a);
        } else {
            System.out.println(b);
        }
    }

    public static void main(String[] args) {
        int x = 10;
        short y = 20;
        printMax(x, y);
        printMax(2 * 7, 3 + 4 + 5);
        // printMax(10L,10);
    }

}
```

```
20
14
```

Funktioniert nicht, da tatsächlicher Parameter vom Typ long übergeben wird

Aufruf mit tatsächlichen Parametern

Ein formaler Parameter wird als lokale Variable angelegt

Der Wert des tatsächlichen Parameters wird berechnet

- Kann auch ein beliebiger Ausdruck sein, der zuerst ausgewertet wird

Der Wert wird in den formalen Parameter **kopiert**

- Es wird implizit eine Zuweisung durchgeführt
- Wird als **Call-by-value** bezeichnet

Implizite Typkonvertierung

- Der Typ eines tatsächlichen Parameters muss gleich oder kleiner als der Typ des formalen Parameters sein
- Beim Aufruf erfolgt, falls benötigt, eine **implizite Typkonvertierung**
- Details siehe Java-Spezifikation

Beispiel (Übergabe bei einfachen Datentypen)

```java
public class ParameterPassingTest {

    private static void test(int value) {
        System.out.println("test before increment: " + value);
        value++;
        System.out.println("test after increment: " + value);
    }

    public static void main(String[] args) {
        int x = 10;
        test(x);
        System.out.println("main: " + x);
        test(x);
        System.out.println("main: " + x);
    }

}
```

```
test before increment: 10
test after increment: 11
main: 10
test before increment: 10
test after increment: 11
main: 10
```

Der Wert von x wird in value gespeichert. Veränderungen an value ändern x nicht! Bei einem neuen Aufruf der Methode werden alle Zuweisungen erneut durchgeführt.

Rückgabe von Werten

- Typ des Rückgabewerts steht vor dem Methodennamen
- Ist void, wenn „Nichts“ zurückgeliefert wird
  - Methode produziert z. B. nur eine Ausgabe (bisherige Beispiele)
- Ergebnis wird durch eine return-Anweisung zurückgegeben
- Ergebnis kann in einem Ausdruck verwendet werden

Beispiel (Methode mit Rückgabewert)

```
public class MethodReturnTest1 {

    private static int calculate(int a, int b) {
        int result;
        result = 2 * (a + b);
        return result;
    }

    public static void main(String[] args) {
        int x, y;
        int result;
        result = calculate(10, 20);
        System.out.println("s = " + result);
        x = result + 10;
        y = result + result;
        System.out.println("x = " + x + ", y = " + y);
        result = calculate(x, y) + y;
        System.out.println("s = " + result);
    }

}
```

Methode mit zwei Parametern **a** und **b** vom Typ **int** und Rückgabetyp **int**

Die lokale Variable **result** ist nur in **calculate** sichtbar

Rückgabe des Inhalts von **result**

result unterscheidet sich von **result** in **calculate**

```
s = 60
x = 70, y = 120
s = 500
```

Beispiel (alternative Variante)

```java
public class MethodReturnTest1 {

    private static int calculate(int a, int b) {
        return 2 * (a + b);
    }


    public static void main(String[] args) {
        int x, y;
        int result;
        result = calculate(10, 20);
        System.out.println("s = " + result);
        x = result + 10;
        y = result + result;
        System.out.println("x = " + x + ", y = " + y);
        result = calculate(x, y) + y;
        System.out.println("s = " + result);
    }
}
```

Kürzere Variante von **calculate**

main liefert nichts zurück – daher void

```
s = 60
x = 70, y = 120
s = 500
```

Rückgabewert

Sollte (muss aber nicht) verwendet werden

- Rückgabewert zuweisen
  - Z. B. sum = *calculate*(10, 20);
- Rückgabewert in einem Ausdruck verwenden
  - Z. B. System.***out***.println(*calculate*(10, 20) * 2);

Methodenaufruf kann auch nur als reine Ausdrucksanweisung (durch Anhängen eines Strichpunkts) vorkommen

- Methoden, die keinen Rückgabewert (void) haben, können nur als Ausdrucksanweisung angeschrieben werden

return-Anweisung

Mehrere return-Anweisungen

- Methode kann an mehreren verschiedenen Punkten vorzeitig beendet werden
- Methoden werden generell kürzer
- Durch mehrere Ausstiegspunkte wird Debugging umständlicher

Leere return-Anweisung

- return;
- Ermöglicht vorzeitiges Beenden einer void-Methode

Beispiel (mehrere return-Anweisungen)

```java
public class PrimeTest {

    private static boolean isPrime(int number) {
        if (number < 2) {
            return false;
        }
        for (int i = 2; i < number; i++) {
            if (number % i == 0) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println(isPrime(1));
        System.out.println(isPrime(4));
        System.out.println(isPrime(31));
        System.out.println(isPrime(99991));
    }

}
```

return-Anweisung bricht die Schleife vorzeitig ab und beendet die Ausführung der Methode!

```
false
false
true
true
```

Beispiel (nur eine return-Anweisung)

```java
public class PrimeTest2 {

    private static boolean isPrime(int number) {
        boolean flag = number >= 2;
        for (int i = 2; i < number && flag; i++) {
            if (number % i == 0) {
                flag = false;
            }
        }
        return flag;
    }


    public static void main(String[] args) {
        System.out.println(isPrime(1));
        System.out.println(isPrime(4));
        System.out.println(isPrime(31));
        System.out.println(isPrime(99991));
    }
}
```

```
false
false
true
true
```

```java
public class PrimeTest2 {

    private static boolean isPrime(int number) {
        boolean flag = number >= 2;
        for (int i = 2; i < number; i++) {
            if (number % i == 0) {
                flag = false;
                break;
            }
        }
        return flag;
    }


    public static void main(String[] args) {
        System.out.println(isPrime(1));
        System.out.println(isPrime(4));
        System.out.println(isPrime(31));
        System.out.println(isPrime(99991));
    }
}
```

Variante mit break

```
false
false
true
true
```

Beispiel (ggT-Berechnung)

```java
public class GCDWithMethods {

    private static int gcd(int a, int b) {
        int first = a;
        int second = b;
        if (first == 0) {
            return second;
        } else {
            while (second != 0) {
                if (first > second) {
                    first -= second;
                } else {
                    second -= first;
                }
            }
        }
        return first;
    }

    public static void main(String[] args) {
        System.out.println(gcd(132, 1908));
        System.out.println(gcd(2640, 150000));
    }
}
```

Klassischer Euklidischer Algorithmus als Methode mit zwei Parametern. Siehe auch: https://de.wikipedia.org/wiki/Euklidischer_Algorithmus

```
12
240
```

Beispiel (ggT-Berechnung)

```
public class GCDWithMethods {

    private static int gcd(int a, int b) {
        if (a == 0) return b;
        else
            while (b != 0)
                if (a > b) a -= b;
                else b -= a;
        return a;
    }

    public static void main(String[] args) {
        System.out.println(gcd(132, 1908));
        System.out.println(gcd(2640, 150000));
    }
}
```

Kürzere Variante:

- Parameter als Variablen benutzen und verändern (eigentlich kein guter Stil)
- Keine Klammern, da immer nur eine Anweisung pro Schachtelungstiefe

Überladen von Methoden

Überladen =

der gleiche Methodenname hat mehrere Bedeutungen

Überladen auch bei Operator +

- + hat unterschiedliche Bedeutung für int, double, String

Überladen

Zwei oder mehrere Methoden dürfen den gleichen Namen haben

Typliste der formalen Parameter muss aber unterschiedlich sein (mindestens eine der folgenden Bedingungen)

- Unterschiedliche Typen
- Unterschiedliche Reihenfolge der Typen
- Unterschiedliche Anzahl der Typen

Unterscheiden mit Hilfe der Signatur

Signatur einer Methode in Java

- Name der Methode
- Liste der Typen der formalen Parameter
  - Die Namen der Parameter spielen keine Rolle, nur die **Typen** und ihre **Reihenfolge**
- Rückgabetyp gehört nicht dazu

Achtung

- Das ist eine eingeschränkte Definition einer Signatur
- Alternative Definitionen folgen in den nächsten Semestern

Beispiel (Überladen von Methoden)

```java
public class OverloadingTest {
    private static void myPrint(int i) {
        System.out.println("Integer: " + i);
    }

    private static void myPrint(double i) {
        System.out.println("Double: " + i);
    }

    private static void myPrint(int i, int j) {
        System.out.println("Integer + Integer: " + i + " " + j);
    }

    private static void myPrint(int i, double j) {
        System.out.println("Integer + Double: " + i + " " + j);
    }

    private static void myPrint(double i, int j) {
        System.out.println("Double + Integer: " + i + " " + j);
    }

    public static void main(String[] args) {
        myPrint(10);
        myPrint(10D);
        myPrint(10, 'a');
        myPrint(10, 10D);
        myPrint(10F, 10);
    }
}
```

```
Integer: 10
Double: 10.0
Integer + Integer: 10 97
Integer + Double: 10 10.0
Double + Integer: 10.0 10
```

Überladen – Verwendung

Methoden haben die gleiche Auswirkung

Unterschiedliche Parameteranzahl oder Typen erfordern unterschiedliche Methoden

Beispiel System.out.println()

- Methode bewirkt immer eine Ausgabe
- Für unterschiedliche Typen werden unterschiedliche Ausgaben produziert

Überladen – Hinweis

Überladen sollte mit Bedacht eingesetzt werden

Schwer lesbar, wenn auf die Details beim Auflösen der Argumente geachtet werden muss

Beispiel aus der Java-API (Math.abs)

```
…
public static int abs(int a) {
    return (a < 0) ? -a : a;
}
…
public static long abs(long a) {
    return (a < 0) ? -a : a;
}
…
public static float abs(float a) {
    return (a <= 0.0F) ? 0.0F - a : a;
}
…
public static double abs(double a) {
    return (a <= 0.0D) ? 0.0D - a : a;
}
…
```

Verkürzte Rückgabe von booleschen Werten

```
private static boolean isEven(int number) {
    if (number % 2 == 0) {
        return true;
    } else {
        return false;
    }
}
```

Korrekt implementiert, aber sehr umständlich!

```
private static boolean isEven(int number) {
    return number % 2 == 0;
}
```

Kurze Implementierung!

IntelliJ-Unterstützung bei Vereinfachungen

```
private static boolean isEven(int number) {
    if (number % 2 == 0) {
        'if' statement can be simplified
        Simplify 'if else'  Alt+Umschalt+Eingabe    More actions...  Alt+Eingabe
    }
        return false;
    }
}
```

```
private static boolean isEven(int number) {
    return number % 2 == 0;
}
```